Here's an example of converting the size comparisons to an RSpec custom matcher:

model/02/spec/support/size_matcher.rb

```
RSpec::Matchers.define :be_of_size do |expected|
  match do |actual|
    actual.size == expected
  end
end
```

The definition starts with a call to RSpec::Matchers.define, passing it the name of the matcher and a block. The block takes an argument that will eventually be the expected value—the value passed to the matcher when called.

Inside the block, the match method is called with a block. That block takes one argument, actual, which is the value passed to expect when the matcher is invoked. The block is expected to do something with the expected and actual values and return true if they match according to the definition of the matcher being written. In this case, you're calling size on the actual value, presumably a Project, and comparing it to the expected value for an equality match. If the matcher takes multiple expected arguments, then the outer block definition should name all the arguments: define :have_sizes do |first, second|.

Remember: The expected value is the value defined by the test, and the actual value is the value defined by the code. Here is the form in which the matcher gets called:

```
expect(actual_value).to be_of_size(expected_value)
```

You can then use this in a test, just like any other matcher:

model/02/spec/models/project_spec.rb

```
it "can calculate total size" do
  expect(project).to be_of_size(10)
  expect(project).not_to be_of_size(5)
end
```

In the first call, 10 is the expected value and project is the actual value. In the second call, not_to is used to negate the matcher, and 5 is the expected value.

There are, of course, additional options. You can call other methods inside the define block to further specify behavior. For instance, you can override the messages RSpec will use when the matcher is displayed: